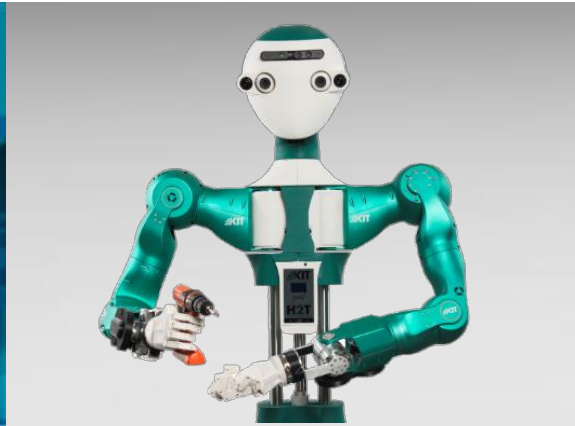


Robotics I: Introduction to Robotics

Chapter 4 – Dynamics

Tamim Asfour and Katja Mombaur

<https://humanoids.kit.edu> <https://hcr.iar.kit.edu/>



Overview

■ Introduction

■ Properties of Dynamic Robot Models

■ Setting up Equations of Motion

■ Euler-Lagrange equations

■ Formulation, Example, Properties of Equations of Motion

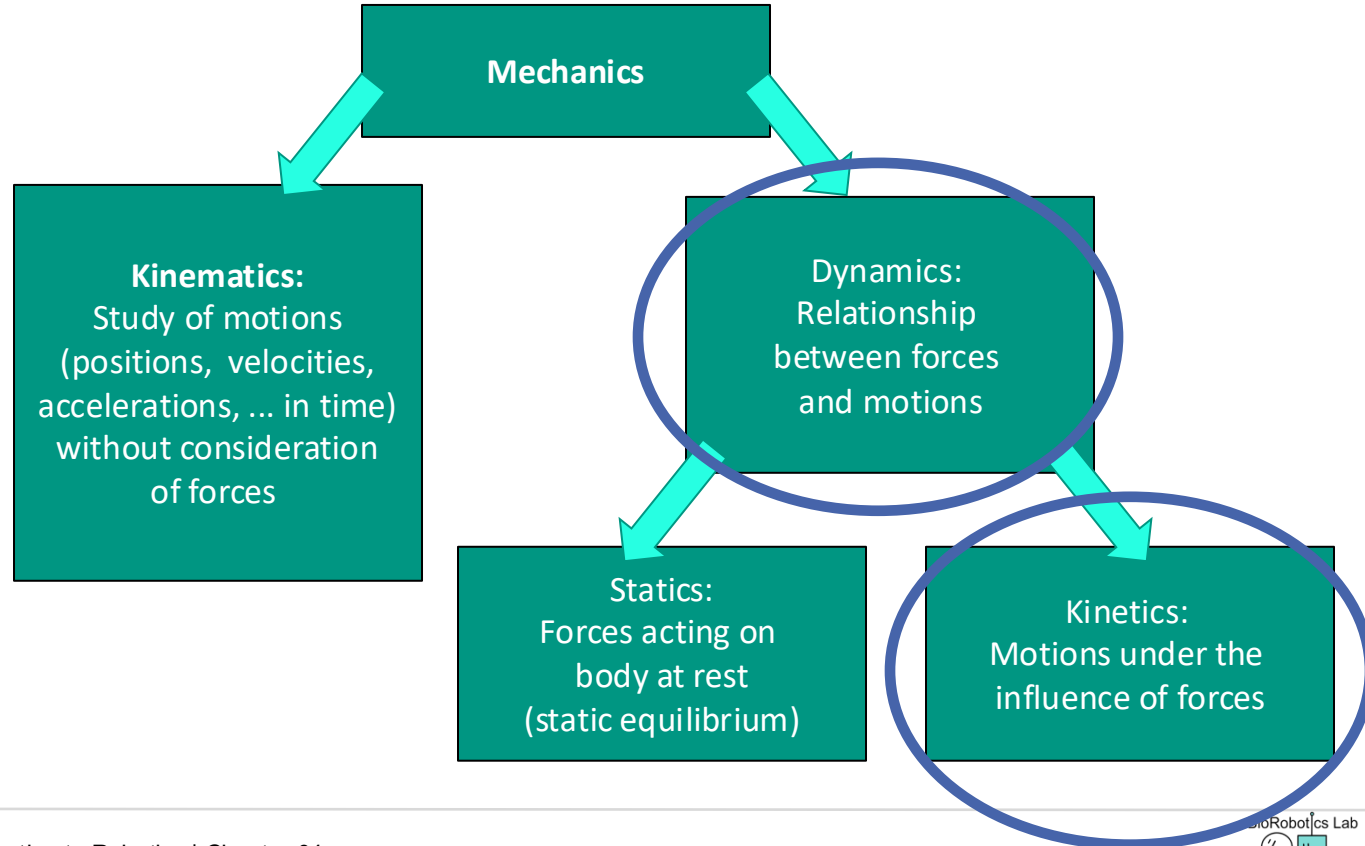
■ Newton-Euler Equations

■ Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools

■ Equations of Motion for Redundant Coordinates

■ Jacobian & Dynamic Manipulability

Overview – Mechanics



Models in Robotics

■ Kinematic Models

Kinematics studies of motion of bodies and systems based only on geometry, i.e. without considering the forces acting on them. The essential concept is the motion. It considers positions (and orientations) and the corresponding velocities, accelerations of bodies.

■ Dynamic Models

Dynamics studies the relationship between the **forces and moments** acting on bodies and the **motions** they produce

■ Geometric Shape Models

Mathematical description of the exact **shape of bodies**.

Overview of contents of lectures

What you learned previously:

- Rigid body mechanics:
 - Mechanical basics
 - Basic definitions for kinematic and dynamic properties
 - Equations of motions (dynamics) of point masses and single extended bodies
- Kinematics of multi-body systems :
 - Forward kinematics
 - Differential kinematics
 - Inverse kinematics

Now:

- **Dynamics of multibody systems**

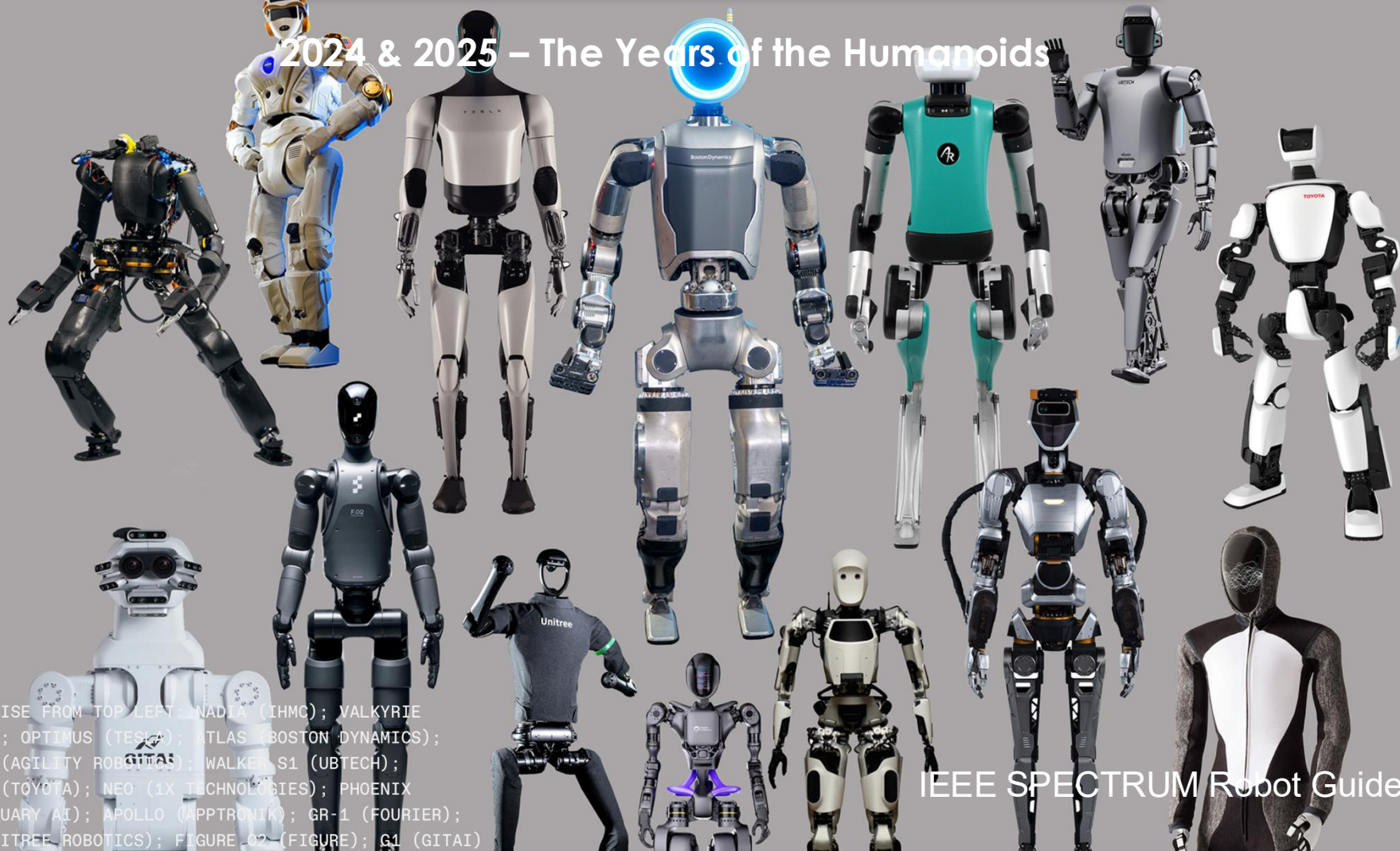
Humanoid robots (e.g. HRP family – AIST / Kawada)



Robots of the same family

but are different in terms of kinematic and dynamic properties

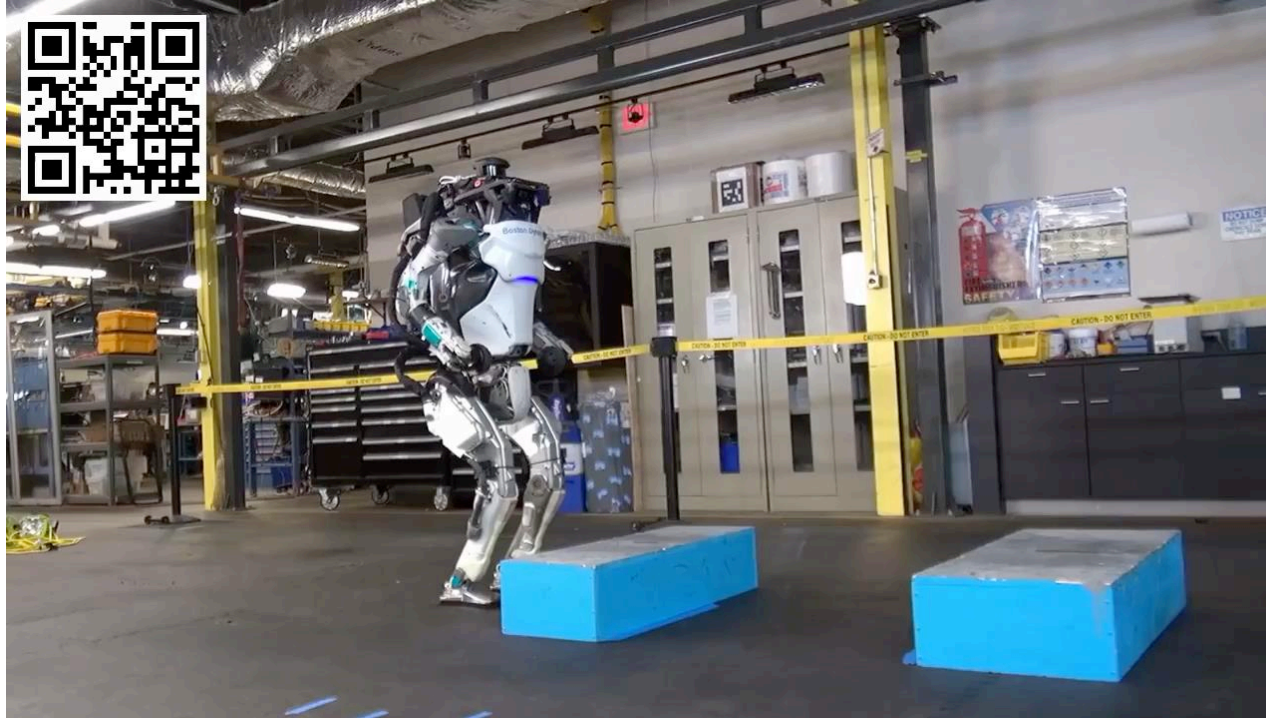
2024 & 2025 – The Years of the Humanoids



CLOCKWISE FROM TOP LEFT: NADIA (IHMC); VALKYRIE (NASA); OPTIMUS (TESLA); ATLAS (BOSTON DYNAMICS); DIGIT (AGILITY ROBOTICS); WALKER S1 (UBTECH); T-HR3 (TOYOTA); NEO (1X TECHNOLOGIES); PHOENIX (SANCTUARY AI); APOLLO (APPTRONIK); GR-1 (FOURIER); H1 (UNITREE ROBOTICS); FIGURE 02 (FIGURE); G1 (GITAI)

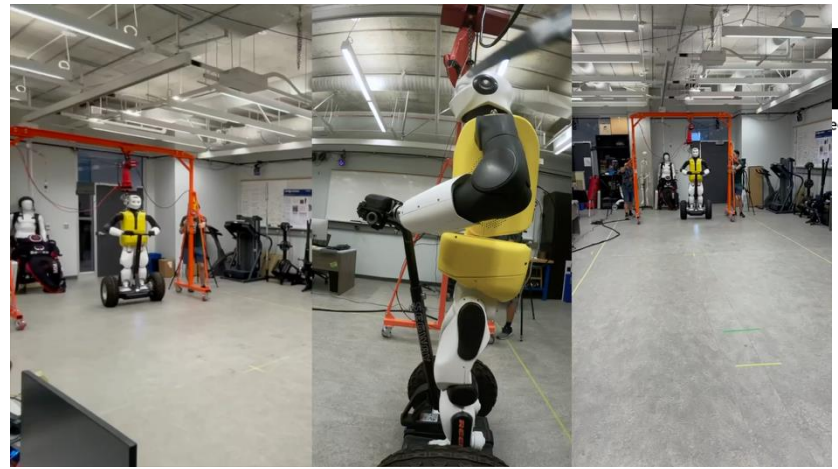
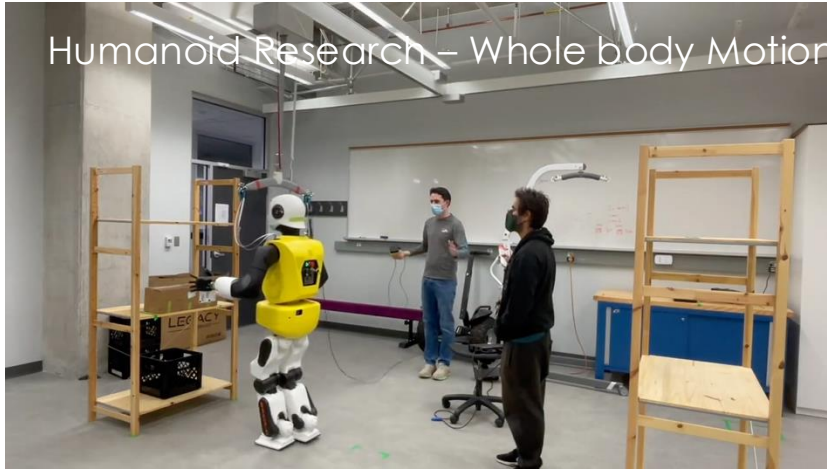
IEEE SPECTRUM Robot Guide

Robot Dynamics – Example Atlas performing somersaults

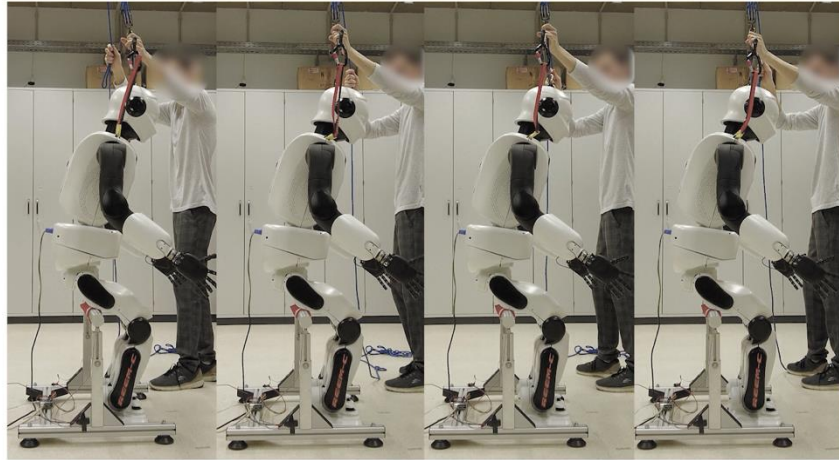


For a long time,
Atlas was the
only really powerful
humanoids,
thanks to its
hydraulic actuators

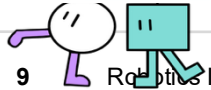
Humanoid Research – Whole body Motion



IT
Technology



Bic



Talos robot (PAL) – torque controlled balancing



Unitree H1 – Dynamic Push recovery



Unitree H1 Dancing



<https://www.youtube.com/watch?v=hWmB4LJ1Ril>

World Humanoids Games



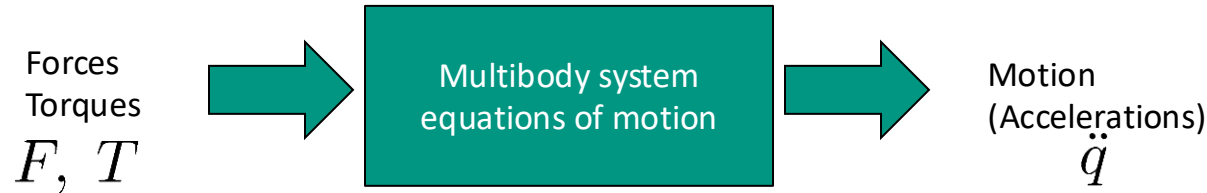
<https://www.youtube.com/watch?v=2SOXsYwuiV4>

... but of course more strength comes with safety concerns

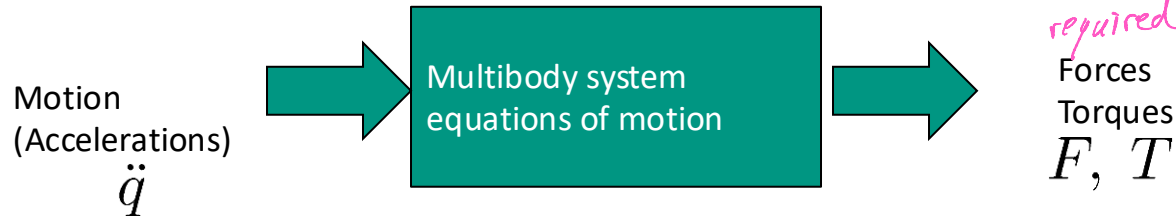


Different tasks in dynamics

- System in a given state $q, \dot{q}$
- Forward dynamics



- Inverse Dynamics

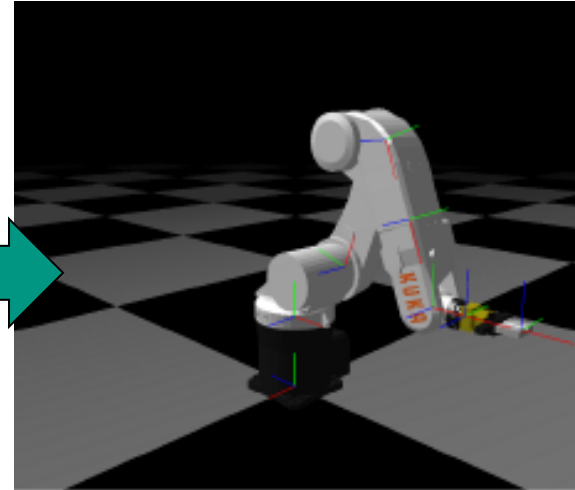


Overview

- Introduction
- **Properties of Dynamic Robot Models**
- Setting up Equations of Motion
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- Equations of Motion for Redundant Coordinates
- Jacobian & Dynamic Manipulability

Modeling of our KUKA robot arm

- Which characteristics of the real robot should the model include?



Modeling of our KUKA robot arm

■ Which characteristics of the real robot should the model include?

Possibilities of motion:
6 degrees of freedom (DOF)

Geometry of the segments

kinematics
dynamics

Generation of motion by motors:
Actuator bounds
(upper and lower limits)



Masses and mass distributions
of the segments (structural
components and motors)

Joint angle bounds
(upper and lower limits)

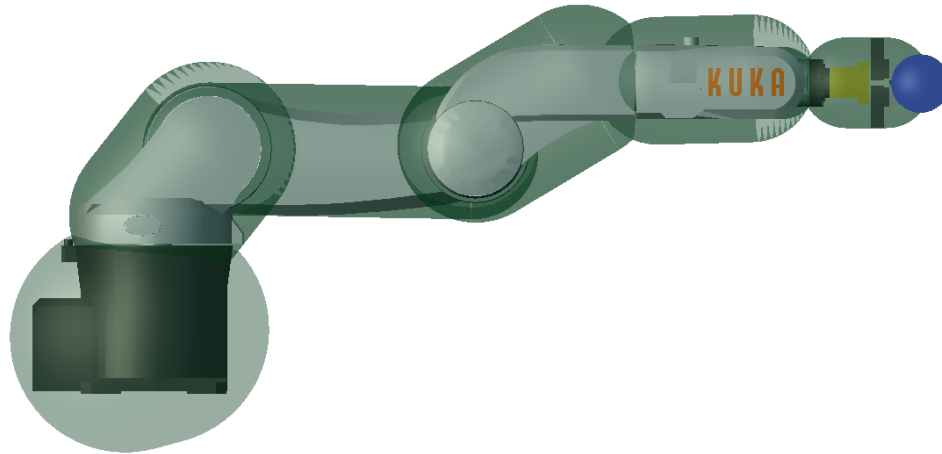
Load limits

Angular velocity bounds
(upper and lower limits)

Modeling of our KUKA robot arm

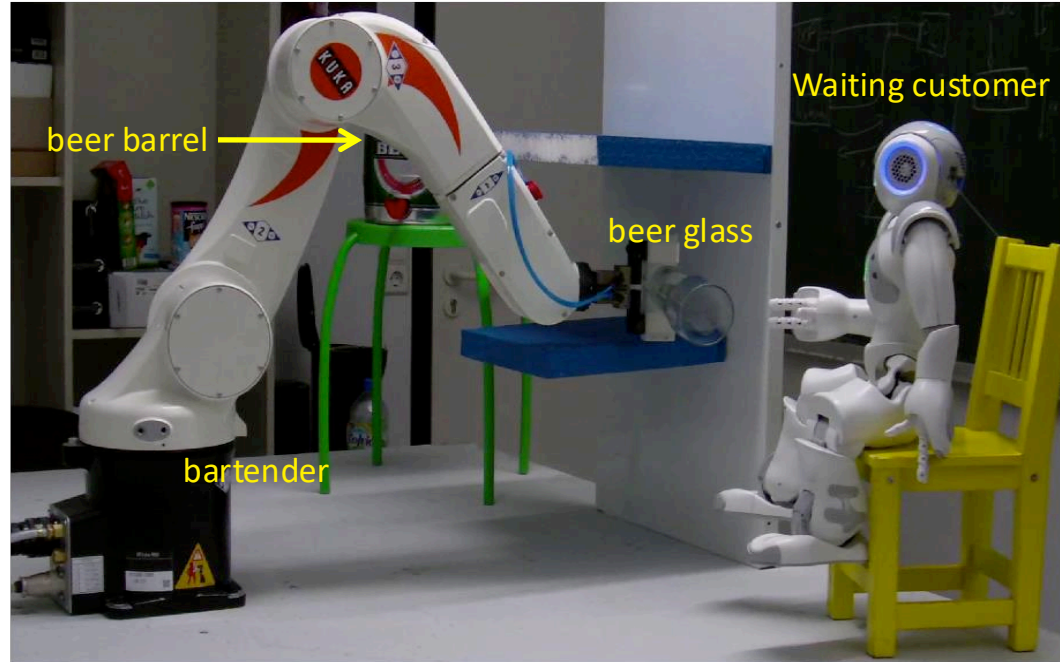
■ Avoiding auto collision:

- simple models of body geometry are needed, e.g. capsules
- otherwise, the complexity will increase dramatically

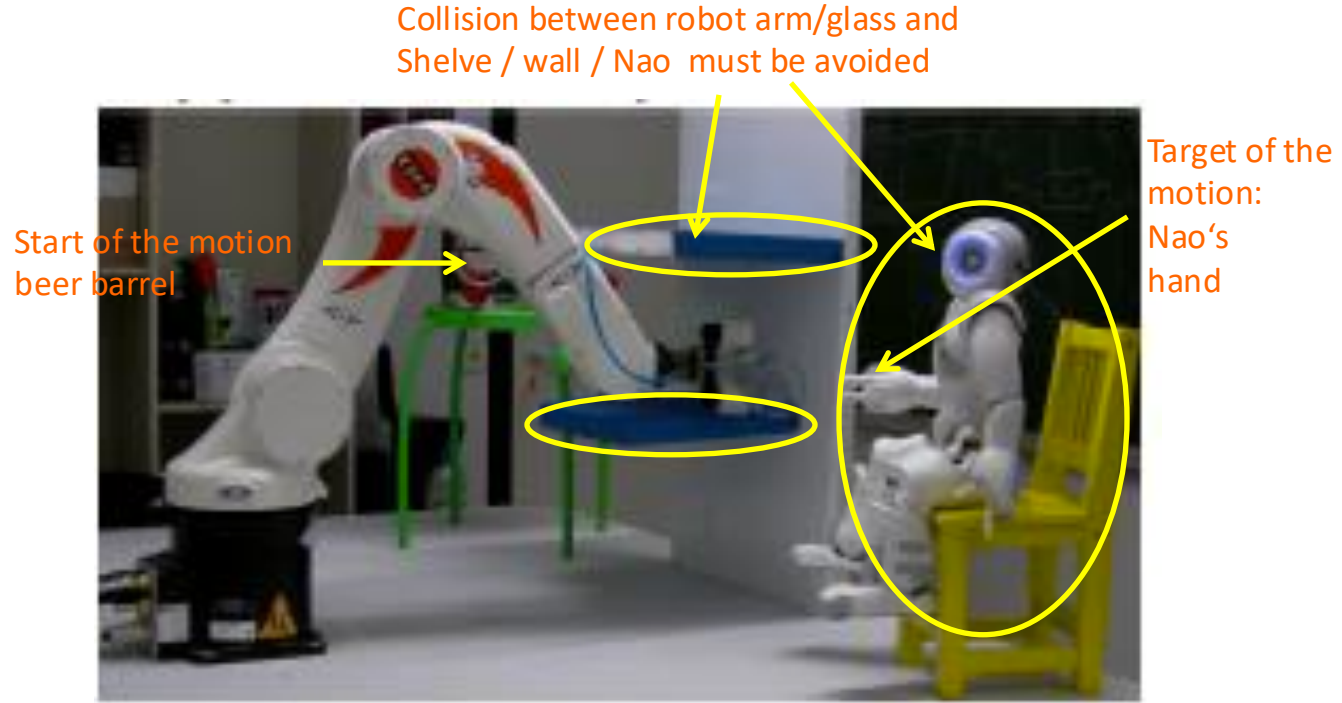


The KUKA Bartender robot

The robot is supposed to serve beer to a customer,
i.e. move the glass from A to B without hitting anything



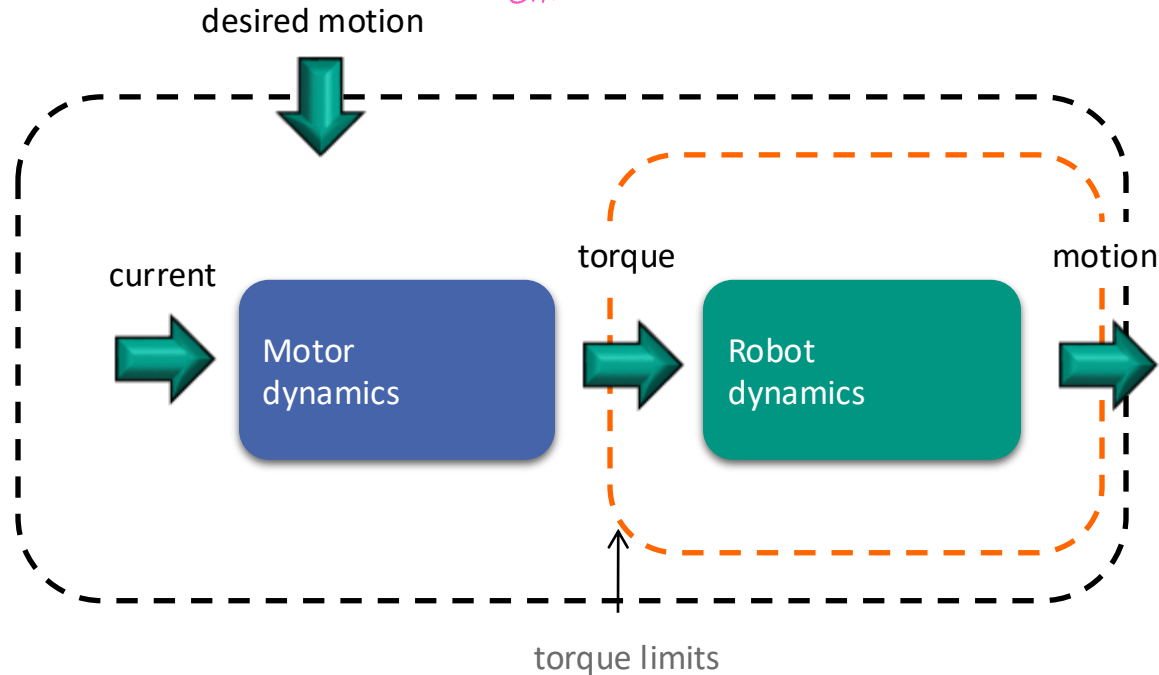
Modeling the environment and the start and end conditions



Modeling of the actuator could be added ..

... to complement the dynamic model of the robot describing the mechanical properties
 (but not necessary)

either control current or torque



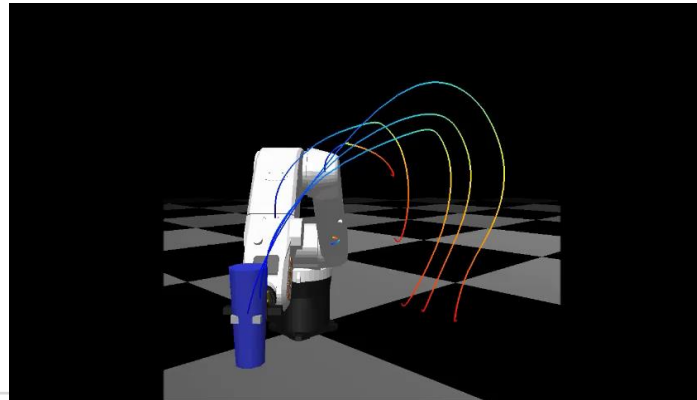
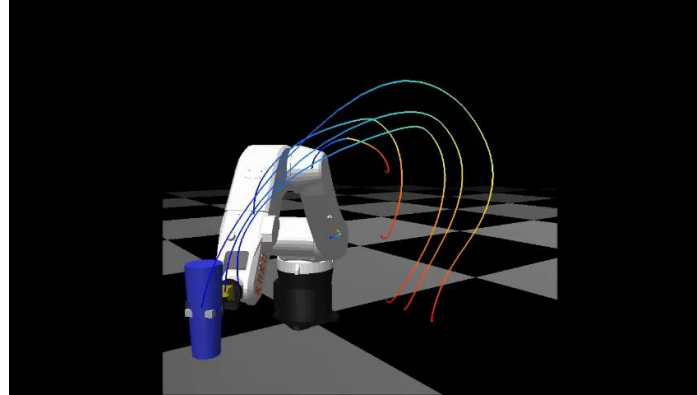
In any case, the motors' masses and inertias have to be taken into account (in addition to structural components), either as part of the segments or separately

Variables of the forward dynamics model

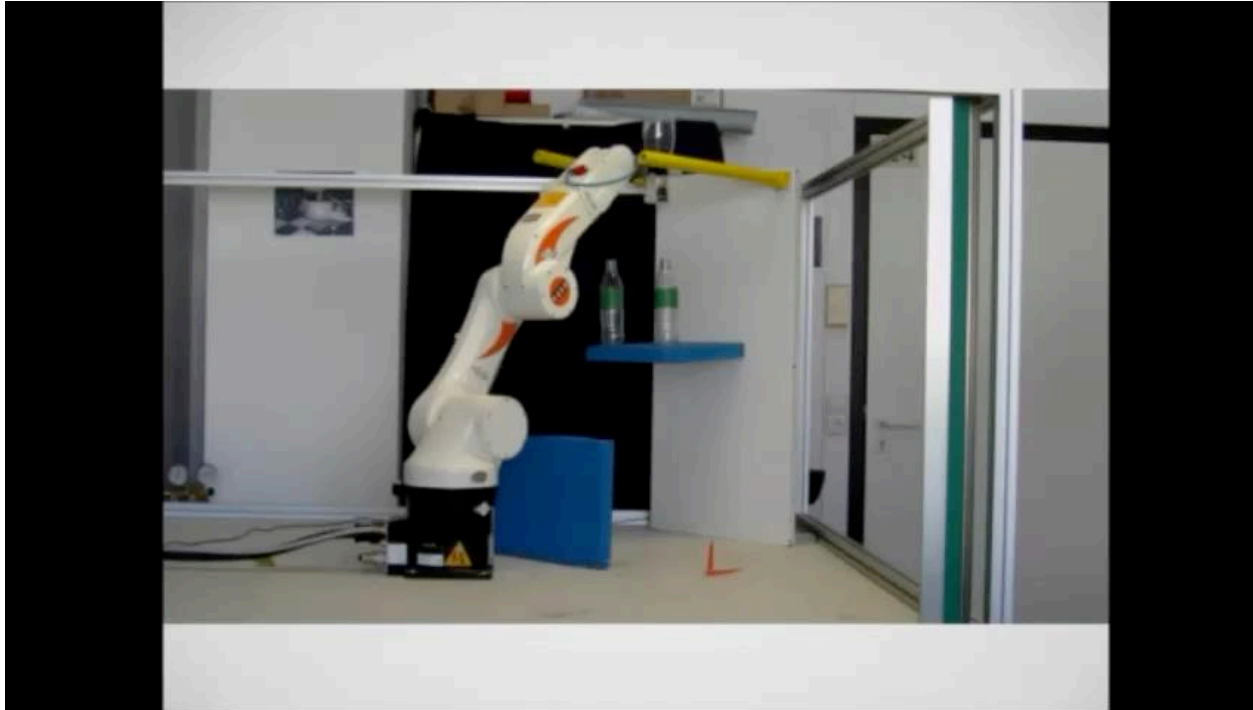
- Inputs / control variables:
Joint torques
- State variables
Joint angles and angular velocities
- Parameters:
Masses, Inertia, Lengths etc.



Optimized motions for the KUKA robot



KUKA in action



Overview

- Introduction
- Properties of Dynamic Robot Models
- **Setting up Equations of Motion**
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- Equations of Motion for Redundant Coordinates
- Jacobian & Dynamic Manipulability

Reminder: Equations of motion for rigid body systems

■ Newton's law (translations)

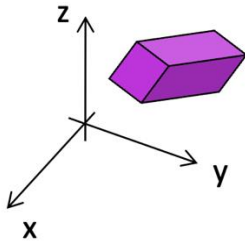
$$F = ma$$

$$F = \sum_{i=1}^{n_T} F_i$$

■ Newton's law (rotations)

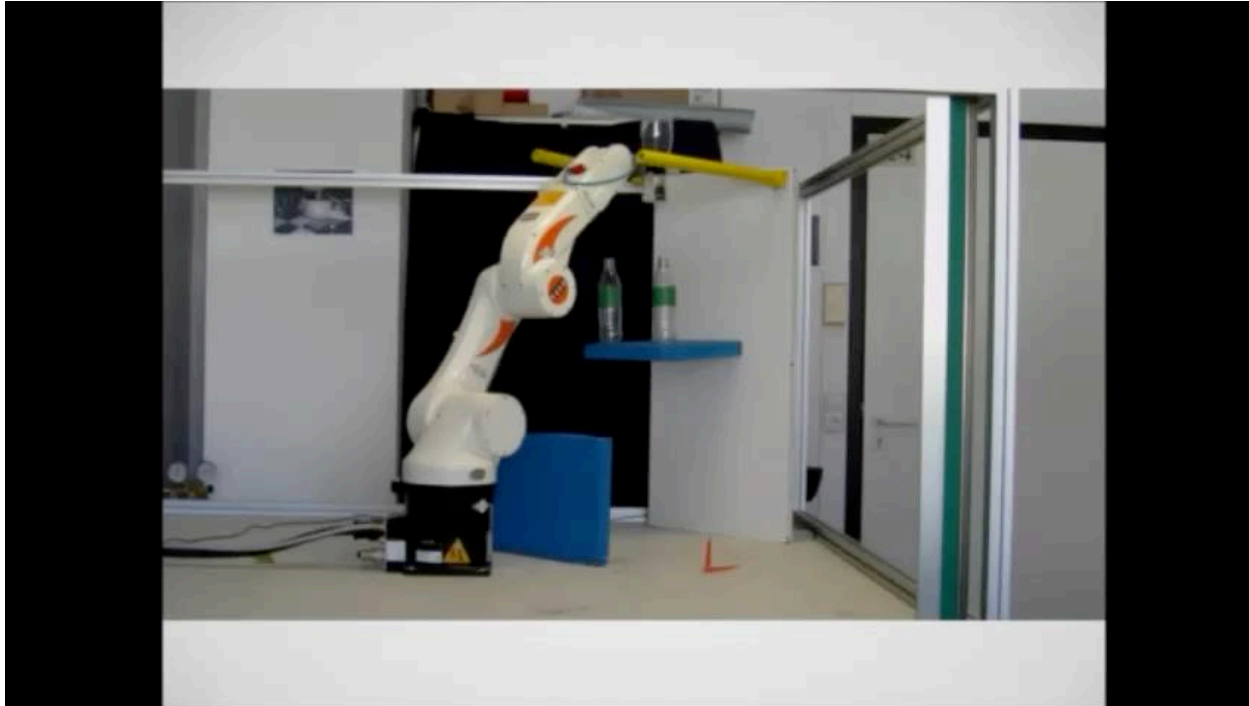
$$T = \Theta \dot{\omega}$$

$$T = \sum_{i=1}^{n_T} T_i$$



How can we set up these equations
for **rigid multibody systems**
and make sure we include all interactions
between the bodies?

How can we set up equations of motion for rigid multibody systems?



How can we set up equations of motion for rigid multibody systems?



Two fundamentally different methods for setting up equations of motion of multibody systems

Lagrange equations / Euler-Lagrange

- Is based on a study of the total energy of the **entire multibody system**
- Is **conceptually very simple**
- Is getting **very complex in practice** already for slightly complicated systems

Newton-Euler / Balance of forces

- Is based on a consideration of the **individual bodies of the multibody systems**
- The **segments are virtually separated** from each other, and Newton's equations are set up individually for each body (free body diagrams)
- **Recursive computations** are possible; **efficient**

Overview

- Introduction
- Properties of Dynamic Robot Models
- Setting up Equations of Motion
- **Euler-Lagrange equations**
 - **Formulation, Example, Properties of Equations of Motion**
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- Equations of Motion for Redundant Coordinates
- Jacobian & Dynamic Manipulability

Generalized coordinates and forces

■ Generalized coordinates q_i

- Set of independent coordinates that allow a complete description of the configuration of a multibody system
- Number generally corresponds to the degrees of freedom of the system
- Coordinates can be positions and angles, but also more general quantities
- $\dot{q}_i$ is the corresponding generalized velocity

■ Generalized forces ξ_i

- Forces / torques etc. corresponding to the generalized coordinates q_i
(not necessarily a physical force - since q_i is not necessarily a position variable)
- ξ_i corresponds to the derivative of the work in the system with respect to q_i

$$\xi_i = \frac{\partial W}{\partial q_i}$$

Lagrange equations of the 2nd kind (Euler-Lagrange method)

■ Lagrange function:

$$\mathcal{L}(q, \dot{q}) = \mathcal{T}(q, \dot{q}) - \mathcal{U}(q)$$

Kinetic Energy Potential Energy

■ (Euler-)Lagrange equation for every component (generalized coordinate = minimal)

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \xi_i$$

$$i = 1, \dots, n$$

ξ_i is the generalized
force corresponding
to coordinate q_i

■ (Euler-)Lagrange equations for all coordinates:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}} \right)^T - \left(\frac{\partial \mathcal{L}}{\partial q} \right)^T = \xi$$

We will discuss the version
for redundant coordinates
shortly

$$B(q)\ddot{q} + n(q, \dot{q}) = \xi$$

Computation of kinetic energy for robot manipulators

Kinetic energy

$$\mathcal{T} = \sum_{i=1}^n (\mathcal{T}_{l_i} + \mathcal{T}_{m_i})$$

Kinetic energy
of each link

Kinetic energy
of each motor

written separately so you
don't forget one
↳ just everything that has
a mass

Kinetic energy of each link

$$\mathcal{T}_{l_i} = \frac{1}{2} m_{l_i} \dot{\mathbf{p}}_{l_i}^T \dot{\mathbf{p}}_{l_i} + \frac{1}{2} \boldsymbol{\omega}_i^T \mathbf{R}_i \mathbf{I}_{l_i}^i \mathbf{R}_i^T \boldsymbol{\omega}_i$$

$$\mathcal{T}_{l_i} = \frac{1}{2} m_{l_i} \dot{\mathbf{q}}^T \mathbf{J}_P^{(l_i)T} \mathbf{J}_P^{(l_i)} \dot{\mathbf{q}} + \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{J}_O^{(l_i)T} \mathbf{R}_i \mathbf{I}_{l_i}^i \mathbf{R}_i^T \mathbf{J}_O^{(l_i)} \dot{\mathbf{q}}$$

Translation &
rotation components

Kinetic energy of motor

$$\mathcal{T}_{m_i} = \frac{1}{2} m_{m_i} \dot{\mathbf{p}}_{m_i}^T \dot{\mathbf{p}}_{m_i} + \frac{1}{2} \boldsymbol{\omega}_{m_i}^T \mathbf{I}_{m_i} \boldsymbol{\omega}_{m_i}$$

Computation of potential energy for robot manipulators

■ Potential energy

$$\mathcal{U} = \sum_{i=1}^n (\mathcal{U}_{\ell_i} + \mathcal{U}_{m_i})$$

Potential energy of each link Potential energy of each motor

← same principle as kinetic energy

■ Potential energy of each link

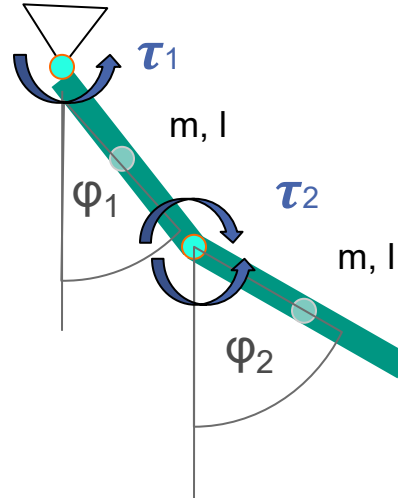
$$\mathcal{U}_{\ell_i} = - \int_{V_{\ell_i}} \mathbf{g}_0^T \mathbf{p}_i^* \rho dV = -m_{\ell_i} \mathbf{g}_0^T \mathbf{p}_{\ell_i}$$

■ Potential energy of each motor

$$\mathcal{U}_{m_i} = -m_{m_i} \mathbf{g}_0^T \mathbf{p}_{m_i}$$

Simple Example: Actuated double pendulum

- Setting up the equations of motion of the double pendulum with the (Euler-)Lagrange method
- Here we assume an idealized system in which the motors have no mass, and only the segments must be considered

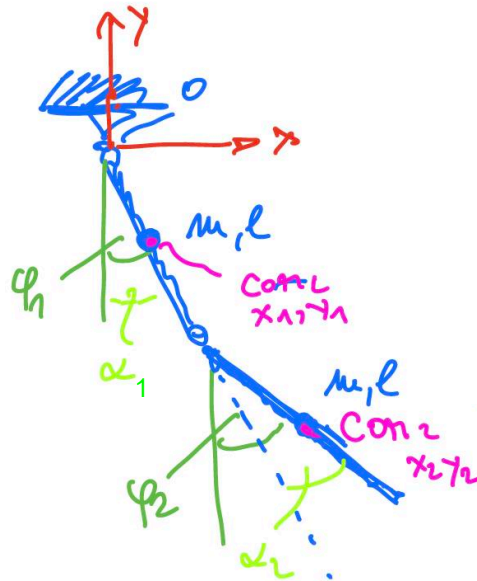


Pendulum segments:
no point masses, but extended
masses (two sticks)

Note: here we do not
distinguish between link
and motor components,
just one mass per segment

Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (1)

■ Sketch with coordinate choices, COS



2DOF, coordinates

$$q = \begin{pmatrix} \phi_1 \\ \phi_2 \end{pmatrix}$$

absolute
ABS ANGLES
WRT
VERTICAL

ALTERNATIVE

$$\alpha_1 = \phi_1$$

$$\alpha_2 = \phi_2 - \phi_1$$

Inertia:

STICK PENDULUM

$$\Theta_{C1/2} = \frac{1}{12} m \ell^2$$

$$\Theta_{O1} = \frac{1}{3} m \ell^2$$

shifted θ_1
into origin

Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (2)

■ Lagrange Function

$$\mathcal{L}(q, \dot{q}) = T(q, \dot{q}) - U(q)$$

■ Kinetic Energy

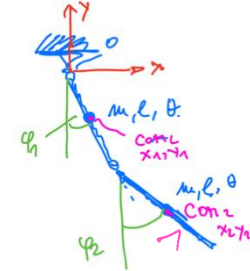
$$\begin{aligned}
 T &= T_1 + T_2 \\
 &\quad \text{1. STICK} \quad \text{2. STICK} \\
 &= \underbrace{\frac{1}{2} \Theta_{O_1} \cdot \dot{\varphi}_1^2}_{\substack{\text{pure} \\ \text{ROTATION} \\ \text{ABOUT POINT O}}} + \underbrace{\frac{1}{2} m (\dot{x}_2^2 + \dot{y}_2^2)}_{\substack{\text{NOT PART OF} \\ \text{COORDINATES} \\ \text{TRANSLATION OF} \\ \text{COM}_2}} + \underbrace{\frac{1}{2} \Theta_{C_2} \cdot \dot{\varphi}_2^2}_{\substack{\text{ROTATION ABOUT} \\ \text{COM}_2}} \quad (*)
 \end{aligned}$$

Stick 1 $\nearrow$

Stick 2 $\nearrow$

com = center of mass

$\dot{x}_2, \dot{y}_2$ NEED TO BE REPLACED:

$$\begin{aligned}
 x_2 &= l \cdot \sin \varphi_1 + \frac{1}{2} l \cdot \sin \varphi_2 \\
 \dot{x}_2 &= l \cdot \cos \varphi_1 \cdot \dot{\varphi}_1 + \frac{1}{2} l \cdot \cos \varphi_2 \cdot \dot{\varphi}_2 \rightarrow \text{SQUARE + INSERT IN } (*)
 \end{aligned}$$


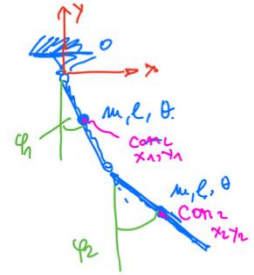
Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (3)

$$y_2 = -l \cdot \cos \varphi_1 - \frac{l}{2} \cdot \cos \varphi_2$$

$$\dot{y}_2 = l \cdot \sin \varphi_1 \cdot \dot{\varphi}_1 + \frac{l}{2} \sin \varphi_2 \cdot \dot{\varphi}_2 \rightarrow \text{SQUARE + INSERT IN (*)}$$

■ Kinetic Energy

$$T(q, \dot{q}) = \frac{2}{3} \cdot m l^2 \cdot \dot{\varphi}_1^2 + \frac{1}{6} m l^2 \dot{\varphi}_2^2 + \frac{1}{2} m l^2 \cdot \dot{\varphi}_1 \cdot \dot{\varphi}_2 \cdot \cos(\varphi_1 - \varphi_2).$$



Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (4)

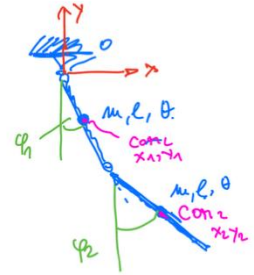
■ Potential Energy

$$\begin{aligned}U &= U_1 + U_2 \\&= m \cdot g \cdot \underbrace{y_1} + m \cdot g \cdot \underbrace{y_2} \dots \\&\quad \text{NOT COORDINATES}\end{aligned}$$

$$\begin{aligned}\text{REDUCE: } y_1 &= -\frac{l}{2} \cos \varphi_1 \\y_2 &\text{ SEE ABOVE}\end{aligned}$$

$$U(q) = m \cdot g \cdot \left(-\frac{l}{2} \cos \varphi_1\right) + m \cdot g \cdot \left(-l \cos \varphi_1 - \frac{1}{2} l \cos \varphi_2\right)$$

$$U(q) = -m \cdot g \cdot \left(\frac{3}{2} \cos \varphi_1 + \frac{1}{2} \cos \varphi_2\right)$$



Now we have both components for Lagrange function.
What should we do with them?

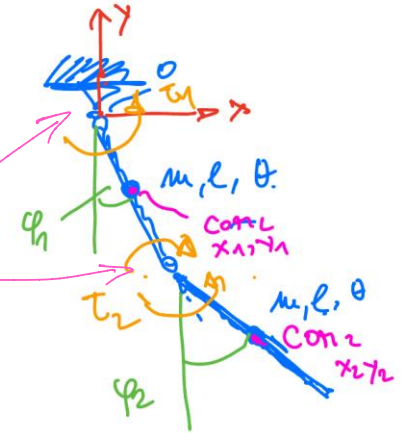
Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (5)

■ Reminder
$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \xi_i \quad i = 1, \dots, n$$

■ Equivalent to:
$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i} \right) - \frac{\partial}{\partial q_i} (\mathcal{T} - \mathcal{U}) = \xi_i \quad (**)$$

■ Generalized forces in case of pendulum

$$\begin{aligned} \xi_1 &= \tau_1 - \tau_2 \\ \xi_2 &= \tau_2 \end{aligned}$$



Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (6)

For φ_1

$$\frac{\partial T}{\partial \varphi_1} = -\frac{1}{2} m \ell^2 \dot{\varphi}_1 \dot{\varphi}_2 \sin(\varphi_1 - \varphi_2)$$

$$\frac{\partial T}{\partial \dot{\varphi}_1} = \frac{1}{3} m \ell^2 \dot{\varphi}_1 + \frac{1}{2} m \ell^2 \dot{\varphi}_2 \cos(\varphi_1 - \varphi_2)$$

Deriving and stuff

$$\begin{aligned} \frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\varphi}_1} \right) &= \frac{1}{3} m \ell^2 \ddot{\varphi}_1 + \frac{1}{2} m \ell^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_2 \\ &\quad + \frac{1}{2} m \ell^2 (-\sin(\varphi_1 - \varphi_2)) \cdot \dot{\varphi}_2 \end{aligned}$$

$$\frac{\partial U}{\partial \varphi_1} = \frac{3}{2} m g \ell \sin \varphi_1$$

Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (7)

INSERT EVERYTHING INTO (***) TO OBTAIN EQNS FOR φ_1

$$\begin{aligned} & \frac{4}{3} m l^2 \ddot{\varphi}_1 + \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_2 \\ & - \frac{1}{2} m l^2 \dot{\varphi}_2^2 \sin(\varphi_1 - \varphi_2) \cdot (\cancel{\dot{\varphi}_1} - \dot{\varphi}_2) \\ & + \frac{1}{2} m l^2 \cancel{\dot{\varphi}_1 \dot{\varphi}_2} \sin(\varphi_1 - \varphi_2) + \frac{3}{2} m g l \cdot \sin \varphi_1 = \tau_1 - \tau_2 \end{aligned}$$

$$\begin{aligned} & \frac{4}{3} m l^2 \ddot{\varphi}_1 + \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \ddot{\varphi}_2 + \frac{1}{2} m l^2 \dot{\varphi}_2^2 \sin(\varphi_1 - \varphi_2) \\ & + \frac{3}{2} m g l \cdot \sin \varphi_1 = \tau_1 - \tau_2 \end{aligned}$$

Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (8)

AND NOW THE SAME FOR φ_2 , with $\xi_2 = \tau_2$:

$$\frac{\partial U}{\partial \varphi_2} = \frac{1}{2} m \cdot g \cdot l \cdot \sin \varphi_2$$

$$\frac{\partial T}{\partial \varphi_2} = \frac{1}{2} m l^2 \ddot{\varphi}_1 \ddot{\varphi}_2 \sin(\varphi_1 - \varphi_2)$$

$$\frac{\partial T}{\partial \dot{\varphi}_2} = \frac{1}{3} m l^2 \dot{\varphi}_2 + \frac{1}{2} m l^2 \dot{\varphi}_1 \cos(\varphi_1 - \varphi_2)$$

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\varphi}_2} \right) = \frac{1}{3} m l^2 \ddot{\varphi}_2 + \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_1 - \frac{1}{2} m l^2 \dot{\varphi}_1 \sin(\varphi_1 - \varphi_2) (\dot{\varphi}_1 - \dot{\varphi}_2)$$

Deriving equations of motion for double pendulum with Euler-Lagrange equations by hand (9)

■ 2nd equation

Writing it into
Matrix

$$\frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_1 + \frac{1}{3} m l^2 \ddot{\varphi}_2 + \frac{1}{2} m g l \sin \varphi_2 - \frac{1}{2} m l^2 \sin(\varphi_1 - \varphi_2) \dot{\varphi}_1^2 = \tau_2$$

Same principle as above

■ Arranging both in matrix-vector form

$$\begin{pmatrix} \frac{4}{3} m l^2 & \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \\ \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) & \frac{1}{3} m l^2 \end{pmatrix} \begin{pmatrix} \ddot{\varphi}_1 \\ \ddot{\varphi}_2 \end{pmatrix} + \begin{pmatrix} + \frac{1}{2} m l^2 \sin(\varphi_1 - \varphi_2) \cdot \dot{\varphi}_2^2 + \frac{3}{2} m g l \sin \varphi_1 \\ - \frac{1}{2} m l^2 \sin(\varphi_1 - \varphi_2) \cdot \dot{\varphi}_1^2 + \frac{1}{2} m g l \sin \varphi_2 \end{pmatrix} = \begin{pmatrix} \tau_1 - \tau_2 \\ \tau_2 \end{pmatrix}$$

Output of Euler-Lagrange: Equations of motion

- From the Lagrange function ...

$$\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}) = \mathcal{T}(\mathbf{q}, \dot{\mathbf{q}}) - \mathcal{U}(\mathbf{q}). \quad \text{with} \quad \mathcal{T} = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{B}(\mathbf{q}) \dot{\mathbf{q}}$$

- and the (Euler-) Lagrange equations ...

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right)^T - \left(\frac{\partial \mathcal{L}}{\partial \mathbf{q}} \right)^T = \boldsymbol{\xi}$$

- ... we obtain equations of motion of the following form

$$\mathbf{B}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{n}(\mathbf{q}, \dot{\mathbf{q}}) = \boldsymbol{\xi}$$

Euler-Larange method
results in explicit
equations of motion
in closed form

Equations of motion – Mass matrix

- **Mass matrix (Inertia matrix) B** – contains all information about mass distribution
 - Is symmetric and positive definite
 - depends in general on the configuration of the system (only for some choices of coordinates it doesn't)

$$B(q) = \sum_{i=1}^n \left(m_{\ell_i} \mathbf{J}_P^{(\ell_i)T} \mathbf{J}_P^{(\ell_i)} + \mathbf{J}_O^{(\ell_i)T} \underbrace{\mathbf{R}_i \mathbf{I}_{\ell_i}^i \mathbf{R}_i^T}_{\text{... in the global system}} \mathbf{J}_O^{(\ell_i)} \right)$$

Mass of body i
Moment of inertia i
in body fixed system

$\mathbf{J}_P^{(\ell_i)}$ $\mathbf{J}_O^{(\ell_i)}$ Jacobians for position and orientation of body i :

$$\dot{\mathbf{p}}_{\ell_i} = \mathbf{J}_{P1}^{(\ell_i)} \dot{q}_1 + \dots + \mathbf{J}_{Pi}^{(\ell_i)} \dot{q}_i = \mathbf{J}_P^{(\ell_i)} \dot{\mathbf{q}}$$

$$\boldsymbol{\omega}_i = \mathbf{J}_{O1}^{(\ell_i)} \dot{q}_1 + \dots + \mathbf{J}_{Oi}^{(\ell_i)} \dot{q}_i = \mathbf{J}_O^{(\ell_i)} \dot{\mathbf{q}}$$

Equations of motion – Nonlinear terms

$$n(\mathbf{q}, \dot{\mathbf{q}}) = \dot{\mathbf{B}}(\mathbf{q})\dot{\mathbf{q}} - \frac{1}{2} \left(\frac{\partial}{\partial \mathbf{q}} (\dot{\mathbf{q}}^T \mathbf{B}(\mathbf{q}) \dot{\mathbf{q}}) \right)^T + \left(\frac{\partial \mathcal{U}(\mathbf{q})}{\partial \mathbf{q}} \right)^T$$

$$= \sum_{j=1}^n \sum_{k=1}^n h_{ijk}(\mathbf{q}) \dot{q}_k \dot{q}_j + g_i(\mathbf{q})$$

←
Torque which is generated
by gravity about i in the
current configuration

with

$$h_{ijk} = \frac{\partial b_{ij}}{\partial q_k} - \frac{1}{2} \frac{\partial b_{jk}}{\partial q_i}$$

$$h_{ijj} \dot{q}_j^2$$

Centrifugal effects on joint i produced
by the velocity of joint j

$$h_{iii} = 0 \quad \text{since} \quad \frac{\partial b_{ii}}{\partial q_i} = 0$$

$$h_{ijk} \dot{q}_j \dot{q}_k$$

Coriolis effect on joint i produced
by the velocities of joint j and k

Equations of motion

- Again, in compact notation, inserting all generalized forces:

$$\underbrace{B(q)\ddot{q}}_{\substack{\text{Inertia term} \\ \text{s.above}}} + \underbrace{C(q, \dot{q})\dot{q}}_{\substack{\text{Nonlinear} \\ \text{term, s. above}}} + \underbrace{F_v \dot{q}}_{\substack{\text{Damping} \\ \text{(viscous friction)} \\ \text{fluid}}} + \underbrace{F_s \operatorname{sgn}(\dot{q})}_{\substack{\text{Friction} \\ \text{(Coulomb friction)} \\ \text{dry sliding}}} + \underbrace{g(q)}_{\substack{\text{Gravity-term, s.above}}} = \underbrace{\tau}_{\substack{\text{Actuator} \\ \text{torques}}} - \underbrace{J^T(q)h_e}_{\substack{\text{Forces and torques between end effector and the environment,} \\ \text{Transformed by corresponding Jacobian}}}$$

where the elements of the matrix C satisfy the following equation:

$$\sum_{j=1}^n c_{ij} \dot{q}_j = \sum_{j=1}^n \sum_{k=1}^n h_{ijk} \dot{q}_k \dot{q}_j$$

Overview

- Introduction
- Properties of Dynamic Robot Models
- Setting up Equations of Motion
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- **Newton-Euler Equations**
 - **Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools**
- Equations of Motion for Redundant Coordinates
- Jacobian & Dynamic Manipulability

Comparison of single and double pendulum



<https://www.youtube.com/watch?v=U39RMUzCjiU>

- Second segment in double pendulum is considerably changing behavior of first segment w.r.t. single pendulum behavior
- Segments are exerting forces on each other

Setting up equations of motion considering individual bodies

1. Basic principles of Newton-Euler method

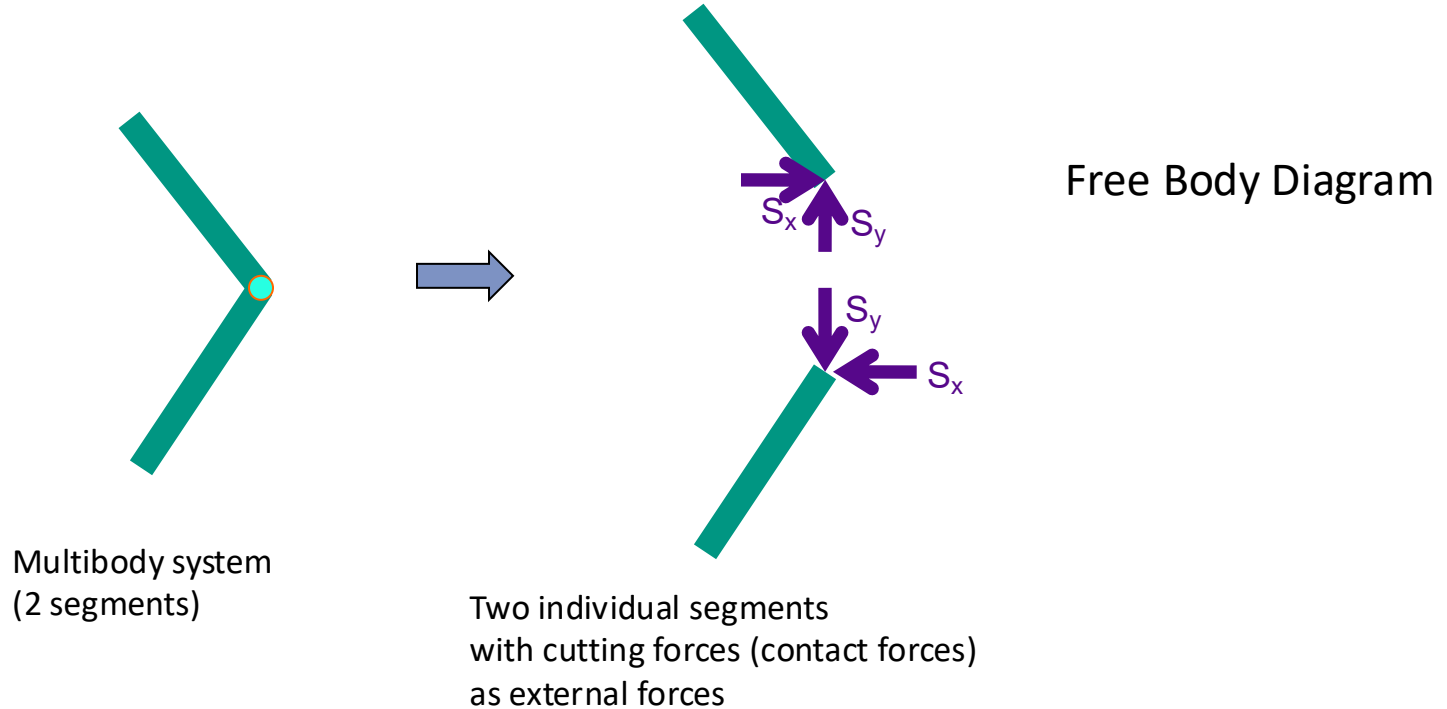
- Principles of free body diagrams and formulation of individual equations of motion
- Direct solution of resulting system (non recursive method):
closed form formulation of equations of motion (only possible for simpler systems)

2. Recursive Newton-Euler

- Keeping all equations and solving them via recursion through multibody system

Cutting forces / Contact forces – How to cut free segments of multibody systems

- Example: Cutting a revolute (rotational) joint in 2D



Cutting forces / torques – How to cut free segments for different joint types

■ Further examples:

- Cutting a spherical joint in 3D (3 rot. DOF)
- Cutting a revolute joint in 3D (1 rot. DOF)
- Cutting a prismatic joint in 2D (1 transl. DOF)

Cutting forces / torques occur in the directions that do NOT correspond to DOFs of the joint

3 cutting forces *← rotation along every axis is allowed*

3 cutting forces, 2 cutting torques *← only rotation along one axis is allowed*

2 cutting forces, 3 cutting torques *← only translation along one axis is allowed*

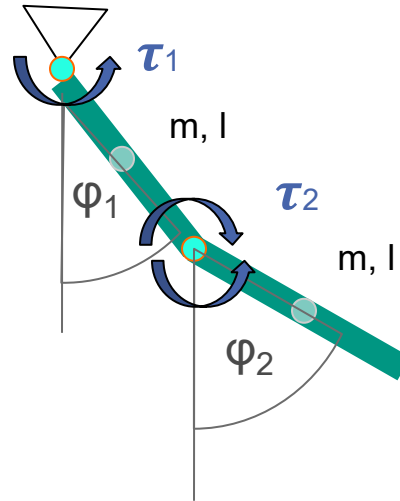
- Actuator forces and torques in the joint act **in addition** to the cutting forces and torques. Actuator forces and torques act in the direction of the degrees of freedom of the joint (or some of them) and remain after cutting the joints.

After all segments are cut free

- Formulate equations of motion separately for each body / segment including
 - In the general case:
 - Newton's equations for translation of center of mass (in all directions)
 - Euler's equations for rotations about center of mass (in all directions)
 - in the case where there is one fixed point (point on the body which is known to not move at all):
 - Euler's equations for rotation about this point (in all directions)
- Formulate kinematic dependencies of the segments through the joints
 - Must still be satisfied after cutting segments free

Simple Example: Actuated double pendulum

- Setting up the equations of motion of the double pendulum with the Newton-Euler approach (free body diagrams)



Deriving equations of motion for double pendulum with Newton-Euler equations by hand (1)

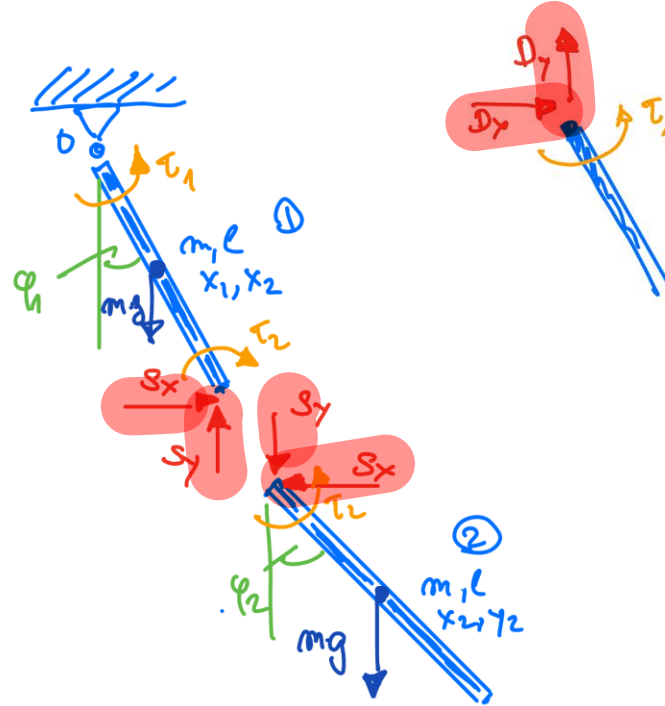
Free Body Diagram

Coordinates

Actuator
torques / moments

Gravity force / weight

Cutting forces
(equivalent to removed
constraint forces)



Joint at ceiling could also be cut free, but not necessary since 0 is a fixed point

$$\theta_{1c} = \theta_{2c} = \frac{1}{12} m l^2$$
$$\theta_{1o} = \frac{1}{3} m l^2$$

Deriving equations of motion for double pendulum with Newton-Euler equations by hand (2)

Body 1: fixed point 0 -> pure rotation about 0 (1 ROT in 2D)

no translation

$$\underbrace{\frac{1}{3} m \ell^2}_{\uparrow} \ddot{\varphi}_1 = \sum T_{0i} = \tau_1 - \tau_2 - m \cdot g \cdot \frac{\ell}{2} \cdot \sin \varphi_1 + S_x \cdot \ell \cdot \cos \varphi_1 + S_y \cdot \ell \cdot \sin \varphi_1$$

Einfache Momente & Kräftegleichgewicht aufstellen (1)

Body 2: translation & rotation. (2 TRANSL + 1 ROT in 2D)

$$m \cdot \ddot{x}_2 = \sum F_{x_i} = -S_x \quad (2)$$

$$m \cdot \ddot{y}_2 = \sum F_{y_i} = -S_y - m \cdot g \quad (3)$$

$$\underbrace{\frac{1}{12} m \ell^2}_{\uparrow} \ddot{\varphi}_2 = \sum T_{c2i} = \tau_2 + S_x \cdot \frac{\ell}{2} \cdot \cos \varphi_2 + S_y \cdot \frac{\ell}{2} \cdot \sin \varphi_2 \quad (4)$$

Deriving equations of motion for double pendulum with Newton-Euler equations by hand (3)

- 4 equations, for 2 DOF
- But there are additional unknowns $S_x, S_y, \ddot{x}_2, \ddot{y}_2$ which need to be eliminated

- Formulate kinematic constraints between $\ddot{x}_2, \ddot{y}_2$ and φ_1, φ_2

$$x = l_1 \cdot \sin(\varphi_1) + \frac{1}{2} l_2 \cdot \sin(\varphi_2)$$

$$\dot{x} = \dots$$
$$\ddot{x} = \dots$$

$$\ddot{x}_2 = +l_1 \cdot \cos \varphi_1 \cdot \ddot{\varphi}_1 - l_1 \cdot \sin \varphi_1 \cdot \dot{\varphi}_1^2 + \frac{1}{2} l_2 \cdot \cos \varphi_2 \cdot \ddot{\varphi}_2 - \frac{1}{2} l_2 \cdot \sin \varphi_2 \cdot \dot{\varphi}_2^2 \quad (5)$$

$$\ddot{y}_2 = l_1 \cdot \sin \varphi_1 \cdot \ddot{\varphi}_1 + l_1 \cdot \cos \varphi_1 \cdot \dot{\varphi}_1^2 + \frac{1}{2} l_2 \cdot \sin \varphi_2 \cdot \ddot{\varphi}_2 + \frac{1}{2} l_2 \cdot \cos \varphi_2 \cdot \dot{\varphi}_2^2 \quad (6)$$

- Insert (5), (6) in (2) and (3)
- Resolve for S_x, S_y
- Use this to eliminate S_x, S_y in (1) and (4)

Deriving equations of motion for double pendulum with Newton-Euler equations by hand (4)

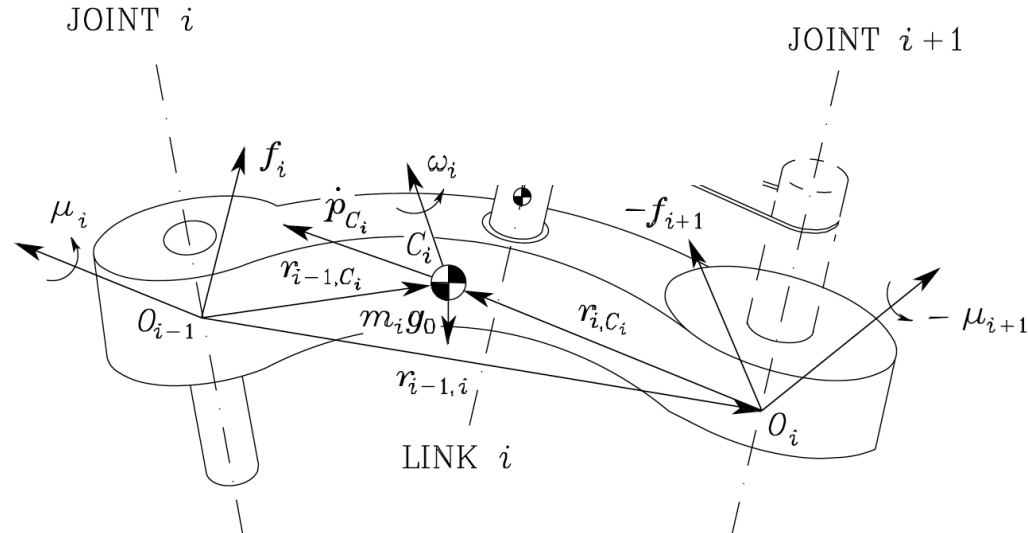
■ Resulting equations of motion

$$\frac{1}{3} m l^2 \ddot{\varphi}_2 + \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_1 = \tau_2 - \frac{1}{2} m g l \sin \varphi_2 + \frac{1}{2} m l^2 \sin(\varphi_1 - \varphi_2) \dot{\varphi}_1^2$$

$$\frac{4}{3} m l^2 \ddot{\varphi}_1 + \frac{1}{2} m l^2 \cos(\varphi_1 - \varphi_2) \cdot \ddot{\varphi}_2 = \tau_1 - \tau_2 - \frac{3}{2} m g l \sin \varphi_1 - \frac{1}{2} m l^2 \sin(\varphi_1 - \varphi_2) \cdot \dot{\varphi}_2^2$$

A general scheme of Newton-Euler

■ Cutting free body i & formulating the balance equations



- All dependencies on rotor inertias and transmission ratios of gears etc. are omitted here for simplicity

Balance equations (equations of motion of segments)

■ Translation (Newton)

$$\mathbf{f}_i - \mathbf{f}_{i+1} + m_i \mathbf{g}_0 = m_i \ddot{\mathbf{p}}_{C_i}$$

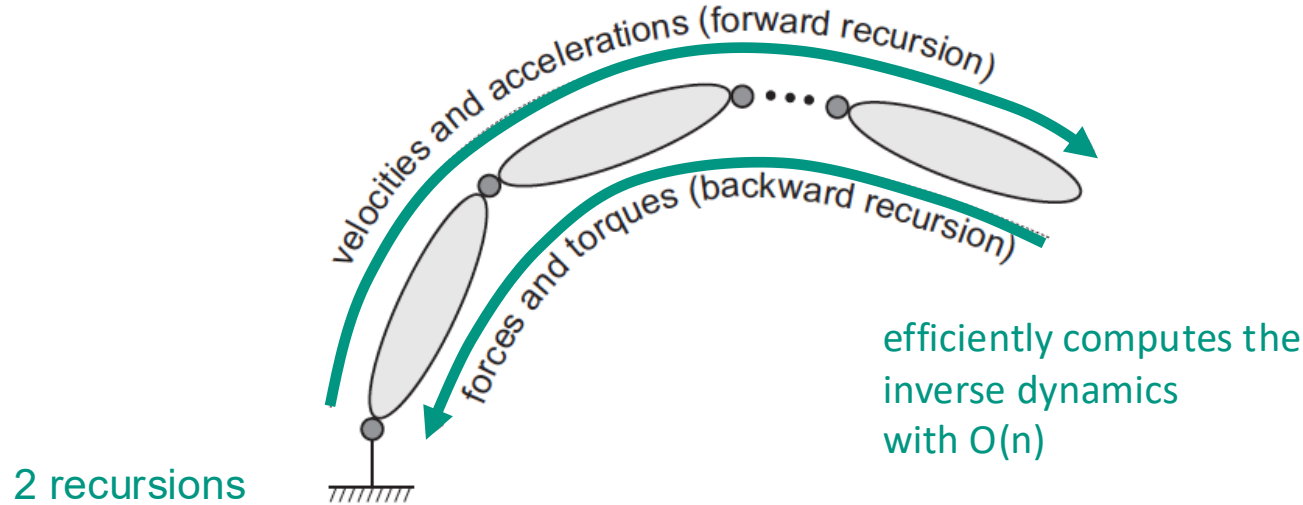
■ Rotation (Euler) (about the segment's center of mass)

$$\boldsymbol{\mu}_i + \mathbf{f}_i \times \mathbf{r}_{i-1, C_i} - \boldsymbol{\mu}_{i+1} - \mathbf{f}_{i+1} \times \mathbf{r}_{i, C_i} = \frac{d}{dt}(\bar{\mathbf{I}}_i \boldsymbol{\omega}_i)$$

with

$$\frac{d}{dt}(\bar{\mathbf{I}}_i \boldsymbol{\omega}_i) = \bar{\mathbf{I}}_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\bar{\mathbf{I}}_i \boldsymbol{\omega}_i)$$

Newton-Euler recursions (RNEA)



- **Forward recursion:** Determines positions, velocities and accelerations of all individual segments (iterating outwards from segment to segment) for given generalized coordinates and their derivatives
- **Backward recursion:** Determines generalized forces as a result of the external forces and inertial forces (iterating inwards)

Segment velocities

■ Relationship between velocities of subsequent segments

■ Angular velocities:

$$\omega_i = \begin{cases} \omega_{i-1} & \text{for a } \textit{prismatic} \text{ joint} \\ \omega_{i-1} + \dot{\vartheta}_i \mathbf{z}_{i-1} & \text{for a } \textit{revolute} \text{ joint} \end{cases}$$

■ Translational velocities

$$\dot{\mathbf{p}}_i = \begin{cases} \dot{\mathbf{p}}_{i-1} + \dot{d}_i \mathbf{z}_{i-1} + \omega_i \times \mathbf{r}_{i-1,i} & \text{for a } \textit{prismatic} \text{ joint} \\ \dot{\mathbf{p}}_{i-1} + \omega_i \times \mathbf{r}_{i-1,i} & \text{for a } \textit{revolute} \text{ joint.} \end{cases}$$

Segment accelerations

- Relationship between the angular accelerations of subsequent segments

$$\dot{\omega}_i = \dot{\omega}_{i-1} \quad \text{for a } \textit{prismatic} \text{ joint}$$

$$\dot{\omega}_i = \dot{\omega}_{i-1} + \ddot{\vartheta}_i \mathbf{z}_{i-1} + \dot{\vartheta}_i \omega_{i-1} \times \mathbf{z}_{i-1} \quad \text{for a } \textit{revolute} \text{ joint}$$

- Relationship between the translational accelerations of subsequent segments

$$\ddot{\mathbf{p}}_i = \ddot{\mathbf{p}}_{i-1} + \ddot{d}_i \mathbf{z}_{i-1} + 2\dot{d}_i \omega_i \times \mathbf{z}_{i-1} + \dot{\omega}_i \times \mathbf{r}_{i-1,i} + \omega_i \times (\omega_i \times \mathbf{r}_{i-1,i})$$

for a *prismatic* joint

$$\ddot{\mathbf{p}}_i = \ddot{\mathbf{p}}_{i-1} + \dot{\omega}_i \times \mathbf{r}_{i-1,i} + \omega_i \times (\omega_i \times \mathbf{r}_{i-1,i}) \quad \text{for a } \textit{revolute} \text{ joint}$$

Forces and torques/moments

■ Forces and torques/moments between bodies

$$\mathbf{f}_i^i = \mathbf{R}_{i+1}^i \mathbf{f}_{i+1}^{i+1} + m_i \ddot{\mathbf{p}}_{C_i}^i$$

$$\begin{aligned} \boldsymbol{\mu}_i^i = & -\mathbf{f}_i^i \times (\mathbf{r}_{i-1,i}^i + \mathbf{r}_{i,C_i}^i) + \mathbf{R}_{i+1}^i \boldsymbol{\mu}_{i+1}^{i+1} + \mathbf{R}_{i+1}^i \mathbf{f}_{i+1}^{i+1} \times \mathbf{r}_{i,C_i}^i \\ & + \bar{\mathbf{I}}_i^i \dot{\boldsymbol{\omega}}_i^i + \boldsymbol{\omega}_i^i \times (\bar{\mathbf{I}}_i^i \boldsymbol{\omega}_i^i) \end{aligned}$$

■ Generalized forces

$$\tau_i = \begin{cases} \mathbf{f}_i^T \mathbf{z}_{i-1} & \text{for a } \textit{prismatic} \text{ joint} \\ \boldsymbol{\mu}_i^T \mathbf{z}_{i-1} & \text{for a } \textit{revolute} \text{ joint} \end{cases}$$

Efficient recursive methods for forward dynamics

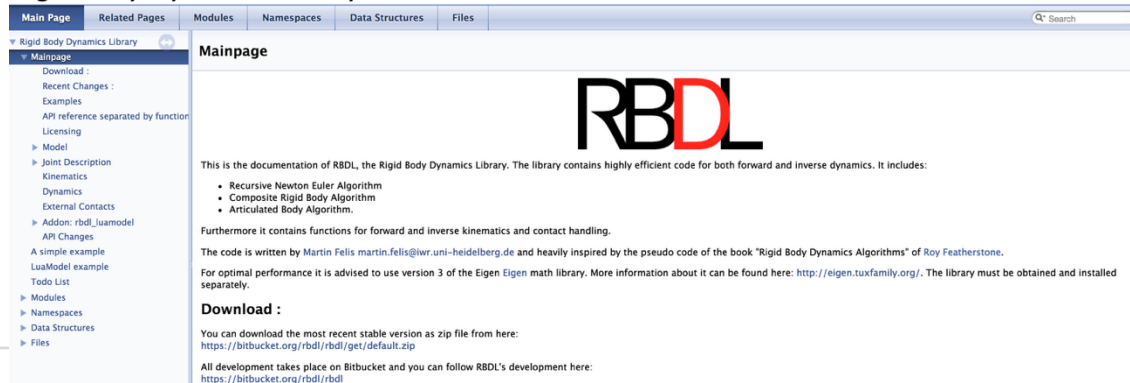
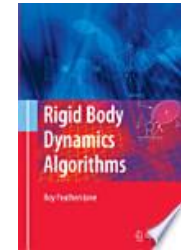
- Articulated body algorithm (Featherstone, 2008)

RBDL – Rigid body dynamics library

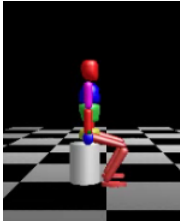
- An efficient tool for setting up models and solving forward and inverse dynamic problems based on Featherstone's algorithms
- Author: Martin Felis, ORB, Heidelberg University (my former group), since 2013
- Contributions by many other ORB team members, full new implementation including visualization and IK tools by Felix Richter (RBDL toolkit) based on former Puppeteer /meshup
- Available on github <https://rbdl.github.io>, <https://github.com/ORB-HD/rbdl-orb>
- Modeling language lua, several example models available



Rigid Body Dynamics Library

Excerpts from a lua model



```

Pelvis = {
    mass = pelvisMass, --
    com = {0., 0., 0.0501 * totalHeight}, --
    inertia = matrix({
        {0.0441 * totalHeight * 0.441 * totalHeight * pelvisMass, 0., 0.},
        {0., 0.041 * totalHeight * 0.041 * totalHeight * pelvisMass, 0.},
        {0., 0., 0.0452 * totalHeight * 0.0452 * totalHeight * pelvisMass}
    })
}

Thigh = {
    mass = thighMass, --
    com = {0., 0., 0.1392 * totalHeight},
    inertia = matrix({
        {thighMass * 0.0804 * 0.0804 * totalHeight * totalHeight, 0., 0.},
        {0., thighMass * 0.0793 * 0.0793 * totalHeight * totalHeight, 0.},
        {0., 0., thighMass * 0.0353 * 0.0353 * totalHeight * totalHeight}
    })
}

Shank = {
    mass = shankMass,
    com = {0., 0., 0.1427 * totalHeight},

    inertia = matrix({
        {shankMass * 0.0693 * 0.0693 * totalHeight * totalHeight, 0., 0.},
        {0., shankMass * 0.0682 * 0.0682 * totalHeight * totalHeight, 0.},
        {0., 0., shankMass * 0.0238 * 0.0238 * totalHeight * totalHeight}
    })
}

Foot = {
    mass = footMass,
    com = {0.0528 * totalHeight, 0, 0.0230 * totalHeight},
    inertia = matrix({
        {footMass * 0.0408 * 0.0408 * totalHeight * totalHeight, 0., 0.},
        {0., footMass * 0.038 * 0.038 * totalHeight * totalHeight, 0.},
        {0., 0., footMass * 0.0189 * 0.0189 * totalHeight * totalHeight}
    })
}

```

```

RightThigh_mesh = {
    dimensions = { 0.1, 0.1, thighLength},
    color = { 0.8, 0.1, 0.1 },
    src = "knubbi_upperleg_R.obj",
    mesh_center = { 0., -0.2, 0.5 * thighLength}
}

```

```

LeftThigh_mesh = {
    dimensions = { 0.1, 0.1, thighLength},
    color = { 0.8, 0.1, 0.1 },
    src = "knubbi_upperleg_L.obj",
    mesh_center = { 0., 0.2, 0.5 * thighLength}
}

```

```

RightShank_mesh = {
    dimensions = { 0.1, 0.1, shankLength},
    color = { 0.9, 0.2, 0.2 },
    src = "knubbi_lowerleg.obj",
    mesh_center = { 0., -0.2, 0.5 * shankLength}
}

```

```

LeftShank_mesh = {
    dimensions = { 0.1, 0.1, shankLength},
    color = { 0.9, 0.2, 0.2 },
    src = "knubbi_lowerleg.obj",
    mesh_center = { 0., 0.2, 0.5 * shankLength}
}

```

```

RightFoot_mesh = {
    dimensions = { footLength, 0.1, footHeight },
    color = { 0.9, 0.2, 0.2 },
    src = "unit_cylinder_medres.obj",
    src = "unit_cube.obj",
    mesh_center = { 0.3 * footLength, -0.2, 0.5 * footHeight}
}

```

```

LeftFoot_mesh = {
    dimensions = { footLength, 0.1, footHeight },
    color = { 0.9, 0.2, 0.2 },
    src = "unit_cylinder_medres.obj",
    src = "unit_cube.obj",
    mesh_center = { 0.3 * footLength, 0.2, 0.5 * footHeight}
}

```

```

-- MidTrunk
{
    name = "midtrunk",
    parent = "pelvis",
    body = MidTrunk,
    joint = joints.rot_y,
    joint_frame = {
        r = {0., 0., pelvisLength},
    },
    visuals = {MidTrunk_mesh},
},

-- UpperTrunk
{
    name = "uppertrunk",
    parent = "midtrunk",
    body = UpperTrunk,
    joint = joints.rot_y,
    joint_frame = {
        r = {0., 0., midTrunkLength},
    },
    visuals = {UpperTrunk_mesh},
},

-- Head
{
    name = "head",
    parent = "uppertrunk",
    body = Head,
    joint = joints.rot_y,
    joint_frame = {
        r = {0., 0., upperTrunkLength},
        r = {0., 0., 0.1183 * totalHeight},
    },
    visuals = {Head_mesh},
},

-- 2UpperArms
{
    name = "twoupperarms",
    parent = "uppertrunk",
    body = TwoUpperArms,
    joint = joints.rot_y,
    joint_frame = {
        r = {0., 0., upperTrunkLength},
    },
    visuals = {
        LeftUpperArm_mesh,
        RightUpperArm_mesh
    },
},

-- 2ForeArms

```

Definition of

- dynamic segment characteristics
- visuals
- joint types

Other rigid body dynamics libraries / simulators

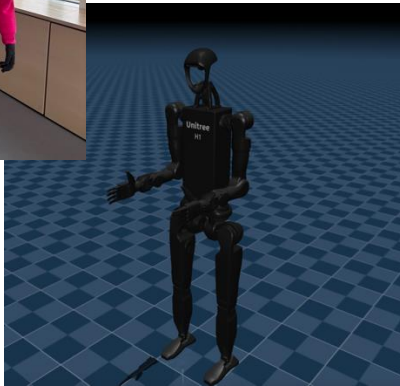
- Pinocchio
- RBdyn
- BioRBD (based on RBDL)

- Bullet
- Gazebo

-

URDF models are often used in robotics

Example Unitree H1-2



```

1  <robot name="h1_2">
2
3      <mujoco>
4          <compiler meshdir="meshes" discardvisual="false"/>
5      </mujoco>
6
7      <!-- [CAUTION] uncomment when convert to mujoco -->
8      <!-- <link name="world"></link>
9      <joint name="floating_base_joint" type="floating">
10         <parent link="world"/>
11         <child link="pelvis"/>
12     </joint> -->
13
14     <link name="pelvis">
15         <inertial>
16             <origin xyz="-0.0004 3.7E-05 -0.046864" rpy="0 0 0"/>
17             <mass value="5.983"/>
18             <inertia ixx="49168.411E-06" ixy="-19.869E-06" ixz="-48.460E-06" iyy="9025.844E-06" iyz="3.431E-06" izz="53155.891E-06"/>
19         </inertial>
20         <visual>
21             <origin xyz="0 0 0" rpy="0 0 0"/>
22             <geometry>
23                 <mesh filename="meshes/pelvis.STL"/>
24             </geometry>
25             <material>
26                 <color rgba="0.1 0.1 0.1 1"/>
27             </material>
28         </visual>
29         <collision>
30             <origin xyz="0 0 0" rpy="0 0 0"/>
31             <geometry>
32                 <mesh filename="meshes/pelvis.STL"/>
33             </geometry>
34         </collision>
35     </link>
36
37     <!-- L Leg -->
38     <link name="left_hip_yaw_link">
39         <inertial>
40             <origin xyz="0 -0.026197 0.006647" rpy="0 0 0"/>
41             <mass value="2.829"/>
42             <inertia ixx="4553.605E-06" ixy="0.005E-06" ixz="0.817E-06" iyy="5688.730E-06" iyz="-345.157E-06" izz="3548.907E-06"/>
43         </inertial>

```

Definition of

- dynamic segment characteristics
- visuals
- joint types

URDF models are often used in robotics

■ Example Unitree H1-2

Definition of

- joint limits (pos, vel)

- sensors

```

54 <joint name="left_hip_yaw_joint" type="revolute">
55 <origin xyz="0 0.0875 -0.1632" rpy="0 0 0"/>
56 <parent link="pelvis"/>
57 <child link="left_hip_yaw_link"/>
58 <axis xyz="0 0 1"/>
59 <limit lower="-0.43" upper="0.43" effort="200" velocity="23"/>
60 </joint>
61 <link name="left_hip_pitch_link">
62 <inertial>
63 <origin xyz="-0.00781 -0.004724 -0.000063" rpy="0 0 0"/>
64 <mass value="2.920"/>
65 <inertia ixx="5594.7E-06" ixy="143.308E-06" ixz="-11.847E-06" iyy="3862.581E-06" iyz="-9.607E-06" izz="4450
66 </inertial>
67 <visual>
68 <origin xyz="0 0 0" rpy="0 0 0"/>
69 <geometry>
70 <mesh filename="meshes/left_hip_pitch_link.STL"/>
71 </geometry>
72 <material>
73 <color rgba="0.1 0.1 0.1 1"/>
74 </material>
75 </visual>
76 <collision>
77 <origin xyz="0 0 0" rpy="0 0 0"/>
78 <geometry>
79 <mesh filename="meshes/left_hip_pitch_link.STL"/>
80 </geometry>
81 </collision>

```

```

826 <!-- IMU -->
827 <link name="imu_link"></link>
828 <joint name="imu_joint" type="fixed">
829 <origin xyz="-0.04452 -0.01891 0.27756" rpy="0 0 0"/>
830 <parent link="torso_link"/>
831 <child link="imu_link"/>
832 </joint>
833
834 <!-- Camera&Lidar -->
835 <link name="camera_link"></link>
836 <joint name="camera_joint" type="fixed">
837 <origin xyz="0.11109 0.01750 0.68789" rpy="0 0.8859291283123216 0"/>
838 <parent link="torso_link"/>
839 <child link="camera_link"/>
840 </joint>
841 <link name="lidar_link"></link>
842 <joint name="lidar_joint" type="fixed">
843 <origin xyz="0.04874 0 0.67980" rpy="0 0.24015730507441985 0"/>
844 <parent link="torso_link"/>
845 <child link="lidar_link"/>
846 </joint>
847 </robot>

```

https://github.com/unitreerobotics/unitree_ros/blob/master/robots/h1_2_description/h1_2_handleless.urdf

Overview

- Introduction
- Properties of Dynamic Robot Models
- Setting up Equations of Motion
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- **Equations of Motion for Redundant Coordinates**
- **Jacobian & Dynamic Manipulability**

Reminder: Equations of motion for non-redundant coordinates

- Above (in Euler Lagrange section) we used the following form:

$$B(q)\ddot{q} + n(q, \dot{q}) = \xi$$

Diagram illustrating the components of the equation of motion:

- Mass matrix (points to $B(q)$)
- Acceleration (points to $\ddot{q}$)
- Nonlinear terms (points to $n(q, \dot{q})$)
- generalized forces (points to ξ)

- The equations of motions have the form of an ODE of 2nd order (in q)
Can be reformulated as first order ODE, introducing $\dot{q}$ as separate variables
- Here the number of coordinates corresponds to the DOFs
- What happens if you use another choice of coordinates?

Equations of motion for redundant coordinates

- Description of the system with redundant coordinates q ($\dim$ of $q > \#$ DOF), which are coupled by constraints:

$$g(q) = 0$$

- Lagrange equations of the 1st kind :

$$\frac{d}{dt} \left(\frac{\partial L(q, \dot{q})}{\partial \dot{q}} \right) - \frac{\partial L(q, \dot{q})}{\partial q} + (G(q)^T \lambda)^T = \xi(t, q, \dot{q})$$

Additional term that considers the forces produced by the constraints (constraints forces)
 $G(q)$ is Jacobian of $g(q)$

- Additionally, the constraints must be satisfied:

$$g(q) = 0$$

Equations of motion for redundant coordinates

- The equations of motion are not any more described by an ODE, as in the case of minimal coordinates, but by a DAE *differential algebraic equation*

- For mechanical systems, this DAE is

$$\begin{aligned} M(q(t), p) \cdot a &= f(q(t), v(t), u(t), p) - G^T(q(t), p)\lambda \\ g_{pos}(q(t), p) &= 0 \end{aligned}$$

*is often easier to describe model like
this and introduce constraints
too also more modular*

- Which index does this DAE have?

Mechanical DAE has got index 3

 $q, \dot{q}, \ddot{q}$

- Can be transformed into index 1 DAE by index reduction (differentiating twice with respect to time)

Algebraic part	$\begin{aligned}\dot{q}(t) &= v(t) \\ \dot{v}(t) &= a(t)\end{aligned}$	Differential part
$\begin{pmatrix} M(q(t), p) & G^T(q(t), p) \\ G(q(t), p) & 0 \end{pmatrix} \begin{pmatrix} a \\ \lambda \end{pmatrix} = \begin{pmatrix} f(q(t), v(t), u(t), p) \\ \gamma(q(t), v(t), p) \end{pmatrix}$		
Invariants	$\begin{aligned}g_{pos} &= g(q(t), p) = 0 \\ g_{vel} &= G(q(t), p) \cdot \dot{q}(t) = 0.\end{aligned}$	

- One further differentiation of the linear system of equations (the algebraic part) would transform the whole system into an ODE (which justifies the statement that the original system was an index 3 DAE)
- Can also be treated in the above form by solving the linear system and inserting the solution (in particular the a) into the differential part

Overview

- Introduction
- Properties of Dynamic Robot Models
- Setting up Equations of Motion
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- Equations of Motion for Redundant Coordinates
- **Jacobian & Dynamic Manipulability**

Reminder: Jacobian Matrix

- Jacobian matrix in differential kinematics – linking velocity spaces

$$\mathbf{x} = f(\boldsymbol{\theta})$$

$$\dot{\mathbf{x}} = J_f(\boldsymbol{\theta}) \cdot \dot{\boldsymbol{\theta}}$$

$$J_f(\boldsymbol{\theta}) = \left(\frac{\partial f_i}{\partial \theta_j}(\boldsymbol{\theta}) \right)_{i,j} = \begin{pmatrix} \frac{\partial f_1}{\partial \theta_1}(\boldsymbol{\theta}) & \dots & \frac{\partial f_1}{\partial \theta_n}(\boldsymbol{\theta}) \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial \theta_1}(\boldsymbol{\theta}) & \dots & \frac{\partial f_m}{\partial \theta_n}(\boldsymbol{\theta}) \end{pmatrix} \in \mathbb{R}^{m \times n}$$

- Properties of the Jacobian - e.g. singularities – have big impact on kinematics
- Jacobian matrix and its singular values are used to quantify (kinematic) manipulability (includes closeness to singularities)

Dynamic manipulability / Force manipulability

- Jacobian can also be used for the analysis of static forces – relating joint torques and forces / torques at end effect effector

- We consider now a sphere in the space of joint torques

$$\boldsymbol{\tau}^T \boldsymbol{\tau} = 1$$

- Using the relationship between torques and end effector force ...

invert

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q})\boldsymbol{\gamma}_e$$

.. the sphere can be mapped to the ellipsoid in the space of end effector forces

$$\boldsymbol{\gamma}_e^T (\mathbf{J}(\mathbf{q})\mathbf{J}^T(\mathbf{q}))\boldsymbol{\gamma}_e = 1$$

Relationship between generalized forces at joints and at the end effector

- We use the term generalized forces since there may be joint torques and forces τ , and also both forces and torques at end effector. γ
- Assume the manipulator / robot arm is at an equilibrium configuration (no motion)
- Application of the principle of virtual work which states that for a system at equilibrium, the total virtual work by all external forces is zero
- Elementary work by joint torques (and forces)

$$dW_{\tau} = \tau^T dq.$$

Work of Joints is the same as Work at End Eff. $\rightarrow$ no motion

- Elementary work by end effector forces and torques $\gamma_e = [f_e^T \quad \mu_e^T]^T$

$$\begin{aligned} dW_{\gamma} &= f_e^T dp_e + \mu_e^T \omega_e dt \\ &= f_e^T J_P(q) dq + \mu_e^T J_O(q) dq \\ &= \gamma_e^T J(q) dq \end{aligned}$$

Relationship between generalized forces at joints and at the end effector

- Condition for static equilibrium:

$$\delta W_\tau = \delta W_\gamma \quad \forall \delta \mathbf{q};$$

with

$$\delta W_\tau = \boldsymbol{\tau}^T \delta \mathbf{q}$$
$$\delta W_\gamma = \boldsymbol{\gamma}_e^T \mathbf{J}(\mathbf{q}) \delta \mathbf{q};$$

which leads to

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q}) \boldsymbol{\gamma}_e$$

Role of Jacobian Matrix in Dynamics

- The Jacobian matrix relates forces and torques at the end effector to the torques in the joints:

$$\boldsymbol{\tau} = \mathbf{J}^T(\mathbf{q})\boldsymbol{\gamma}_e$$

- The following problems can be solved with this relation:
 - Given forces/torques at the end effector, which forces/ torques must act in the joints to resist them?
 - Given the forces/torques in the joints, which resulting forces and torques act on the (fixed) end-effector?

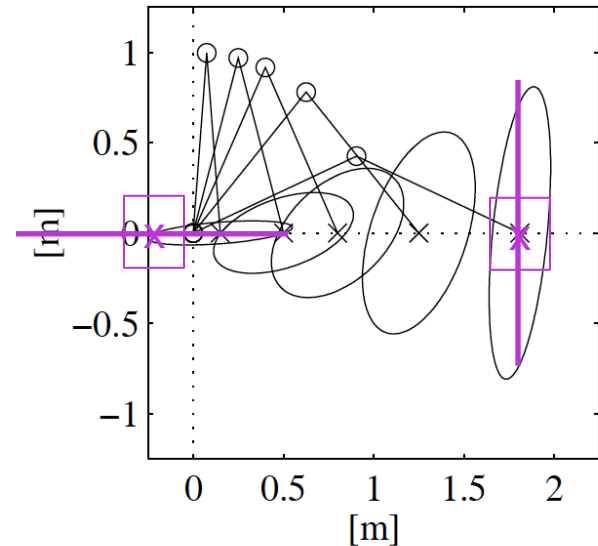
Reminder: Manipulability (Kinematic Manipulability)

- The **manipulability ellipsoid (velocity manipulability ellipsoid)** shows us **how close the manipulator is to a singularity**
- When the configuration approaches a singularity, the ellipse collapses to a line segment
- Using the major and minor principal semi-axes of the manipulability ellipsoid

$$\ell_{\max} / \ell_{\min}$$

the closeness to a singularity can be evaluated

- If ellipsoid is close to circle (ratio close to 1), the tip can easily move in all directions
- If ratio goes to infinity, configuration is close to singularity



Now: Dynamic Manipulability

- Based on the force relationship between end effector and joints established above, we can also define manipulability in the context of forces:
- We consider the unit sphere in the space of joint torque

$$\boldsymbol{\tau}^T \boldsymbol{\tau} = 1$$

- This results in

$$\boldsymbol{\gamma}_e^T (\mathbf{J}(\mathbf{q}) \mathbf{J}^T(\mathbf{q})) \boldsymbol{\gamma}_e = 1$$

which is defined as the force manipulability ellipsoid

This ellipsoid characterizes the end-effector forces that can be generated with the given set of joint torques.

Note that $\mathbf{J}(\mathbf{q})$, so this ellipsoid is valid for the manipulator in a given posture $\mathbf{q}$

Comparison kinematic and dynamic manipulability ellipsoids

Kinematic

Dynamic

Joint space

$$\dot{q}^T \dot{q} = 1$$

$$\tau^T \tau = 1$$

End effector space

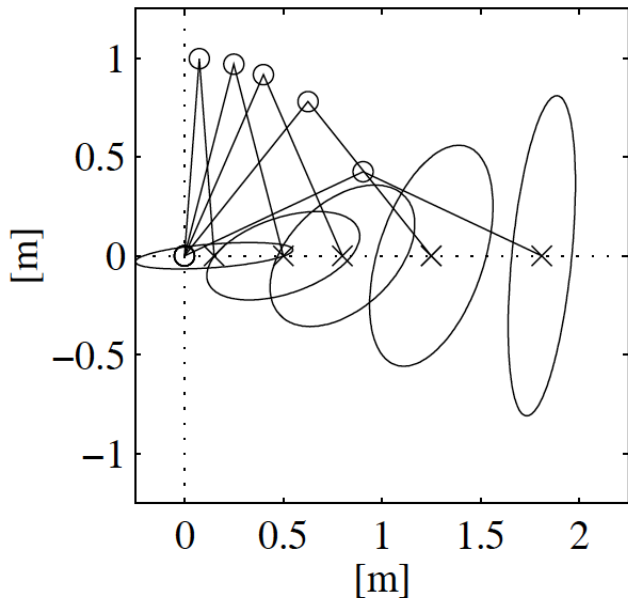
$$v_e^T \left(J(q) J^T(q) \right)^{-1} v_e = 1$$

$$\gamma_e^T \left(J(q) J^T(q) \right) \gamma_e = 1$$

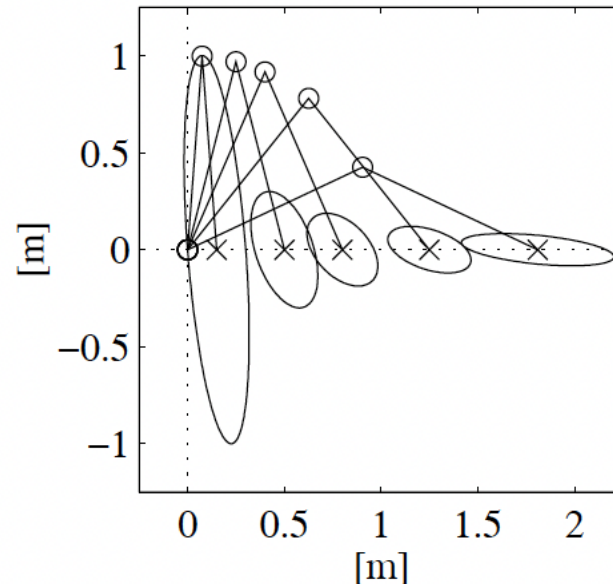
Inverse

2D example: manipulability of 2 DOF manipulator arm

Kinematic Manipulability



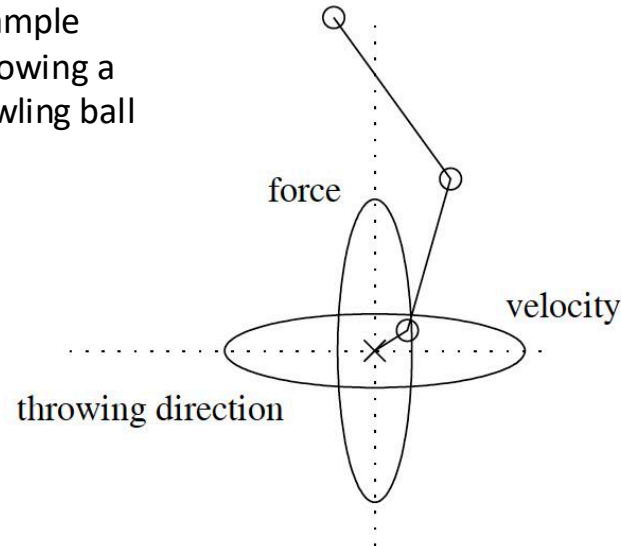
Dynamic Manipulability



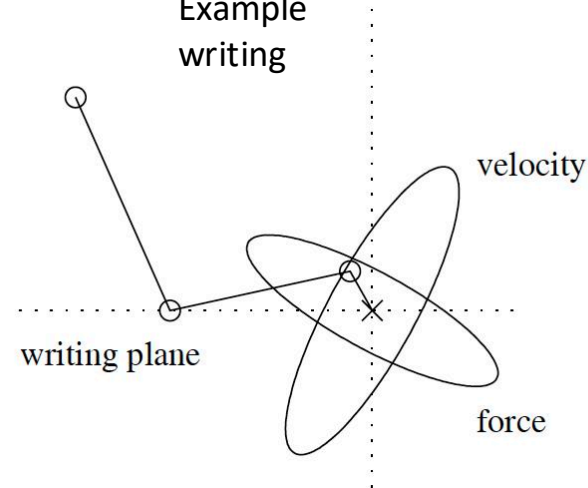
2D example: manipulability of 3 DOF arm

- a simplified human-like arm (upper & lower arm, hand) performing 2 different tasks in 2D end effector space (horizontal and vertical)

Example
throwing a
bowling ball



Example
writing



Summary

- Introduction
- Properties of Dynamic Robot Models
- Setting up Equations of Motion
- Euler-Lagrange equations
 - Formulation, Example, Properties of Equations of Motion
- Newton-Euler Equations
 - Formulation, Example, Recursive Newton-Euler Algorithms, Modeling tools
- Equations of Motion for Redundant Coordinates
- Jacobian & Dynamic Manipulability